TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP LỚN TRIỂN KHAI VÀ QUẢN TRỊ HỆ THỐNG PHẦN MỀM

Đề tài 30:

HỆ THỐNG QUẢN LÝ ĐỘI XE
LAM FLEET

Sinh viên thực hiện: Nguyễn Thị Lam
Mã số sinh viên (MSSV): DTC245200411
Lớp / Khóa: CNTT K23D
Giảng viên hướng dẫn: Vũ Việt Dũng

Thái Nguyên, 2026

MỤC LỤC

Chương 1 Tổng quan kiến trúc và môi trường

1.1. Mục tiêu và phạm vi hệ thống 3

1.2. Kiến trúc và môi trường phát triển 4

1.3. Quản lý mã nguồn và quy trình phát triển 5

Chương 2 Phát triển ứng dụng database và reverse proxy

2.1. Phân tích chức năng và tổ chức ứng dụng 6

2.2. Thiết kế cơ sở dữ liệu 7

2.3. Ràng buộc dữ liệu và quy tắc kiểm tra 8

2.4. Phát triển giao diện và giao tiếp HTTP 9

2.5. Transaction và xử lý đồng thời 10

2.6. Xây dựng vòng đời chuyển đi và bảo dưỡng 11

2.7. Phát triển xác thực và quản lý phiên 12

2.8. Đóng gói ứng dụng và cấu hình reverse proxy 13

Chương 3 Giám sát và quản lý log

3.1. Quản trị giám sát bằng Prometheus và Grafana 14

3.2. Quản lý log và truy vết sự cố 15

Chương 4 Quản trị bảo mật và đánh giá

4.1. Phân quyền và gia cố bảo mật 16

4.2. Quản trị dữ liệu và sao lưu phục hồi 17

4.3. Quản lý thay đổi và xử lý sự cố 18

4.4. Kiểm thử và đánh giá kết quả 19

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 20

Báo cáo trình bày kiến trúc và môi trường, phát triển ứng dụng, giám sát, quản trị dữ liệu và bảo mật. Các kết quả được đối chiếu với mã nguồn và hồ sơ nghiệm thu của Lam Fleet để làm rõ cách thiết kế, triển khai và duy trì hệ thống.

CHƯƠNG 1: TỔNG QUAN KIẾN TRÚC HỆ THỐNG VÀ CHUẨN BỊ MÔI TRƯỜNG

1.1. Mục tiêu và phạm vi hệ thống

Lam Fleet là ứng dụng quản lý đội xe dành cho một đơn vị vận tải ở quy mô bài thực hành. Hệ thống giúp quản trị viên tổ chức hồ sơ xe và tài xế, lập chuyến đi, sắp xếp bảo dưỡng và tra cứu lịch sử. Một lịch phù hợp phải đồng thời thỏa điều kiện về phương tiện, người thực hiện và khoảng thời gian, thay vì chỉ lưu được thông tin biểu mẫu.

Mục tiêu phát triển là chuyển các quy tắc điều phối thành hành vi nhất quán trong mã nguồn và cơ sở dữ liệu. Mục tiêu quản trị là có thể dựng lại hệ thống, kiểm soát quyền, theo dõi lỗi và khôi phục dữ liệu bằng quy trình có thể kiểm chứng. Vì vậy, tính đúng đắn của một thao tác và khả năng vận hành sau triển khai được xem xét cùng nhau.

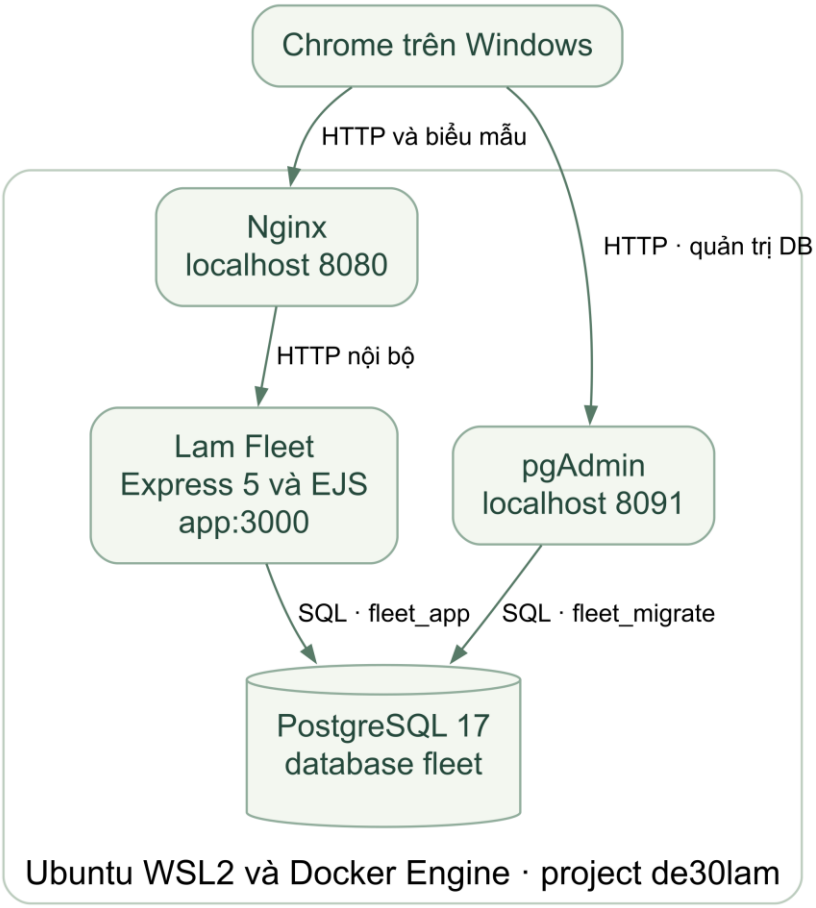
Nhóm yêu cầu	Năng lực cần đáp ứng
Hồ sơ	Mã xe, biển số, giấy phép duy nhất; theo dõi trạng thái và kilomet
Điều phối	Lập, sửa và hủy lịch dự kiến; chặn trùng xe hoặc tài xế
Bảo dưỡng	Ghi khoảng thời gian, chi phí và kết quả; bảo toàn lịch sử
Phát triển	Phân tách giao diện, HTTP, nghiệp vụ và truy cập dữ liệu
Quản trị	Phân quyền, quản lý secret, sao lưu và kiểm soát thay đổi
Đánh giá	Kiểm thử nghiệp vụ, tranh chấp đồng thời và khởi tạo mới

Phạm vi hiện tại có một quản trị viên chính, giao diện tiếng Việt, giờ hiển thị Việt Nam và chi phí VND. Không có đăng ký công khai hay phân quyền nhiều nhóm người dùng. GPS, thanh toán, tối ưu tuyến và ứng dụng di động được dành cho giai đoạn phát triển sau, do chúng đòi hỏi thêm dữ liệu, tích hợp và cơ chế bảo vệ riêng.

Dữ liệu khởi tạo gồm 8 xe, 6 tài xế, 10 chuyến và 5 phiếu bảo dưỡng. Tên, điện thoại và giấy phép mẫu là dữ liệu minh họa. Kết quả nghiệm thu trong báo cáo dựa trên mã nguồn và các tệp minh chứng của project de30lam, không suy rộng thành cam kết về tải lớn hoặc tính sẵn sàng cao trong môi trường sản xuất.

1.2. Kiến trúc và môi trường phát triển

Ứng dụng được tổ chức theo các module trong cùng một tiến trình Express, còn database, reverse proxy và công cụ quản trị chạy ở các container riêng. Đây là ứng dụng nguyên khối có phân lớp, kết hợp các dịch vụ hạ tầng độc lập; việc tách container không đồng nghĩa mỗi chức năng nghiệp vụ là một microservice.



Hình 1. Kiến trúc triển khai Lam Fleet và ranh giới truy cập

Trình duyệt truy cập Nginx ở localhost:8080. Nginx chuyển tiếp tới app:3000 trong mạng nội bộ, còn app dùng SQL để truy cập PostgreSQL. pgAdmin là đường quản trị database riêng, không đi qua giao diện nghiệp vụ. Các lớp frontend, database và monitoring có mạng riêng để giảm bề mặt truy cập.

Môi trường thực hiện dùng Docker Engine trong Ubuntu WSL2, Node.js 24, Express 5, EJS và PostgreSQL 17. Mã nguồn chính nằm ở /home/hai/projects/de30-quan-ly-doi-xe. Khi chuyển sang máy khác, người quản trị cần kiểm tra Docker, Compose, quyền thư mục và cổng trước khi chạy script chuẩn bị.

1.3. Quản lý mã nguồn và quy trình phát triển

Repository riêng tư DTC245200411/dtc245200411-de30-quan-ly-doi-xe quản lý cả mã ứng dụng lẫn cấu hình hạ tầng. Tác giả Git là Nguyễn Thị Lam; tài khoản GitHub, MCP và credential dùng khi đẩy mã đã được xác minh đúng DTC245200411. Nhánh bàn giao trên GitHub là main, giúp việc clone và đối chiếu phiên bản có một điểm tham chiếu rõ ràng.

Mốc	Nội dung đã ghi nhận	Commit
Ứng dụng	Website Fleet, PostgreSQL, pgAdmin và Nginx	70b2173
Giám sát	Prometheus, exporter, cAdvisor và Grafana	8d2fa4a
Bàn giao	Log tập trung, hardening, phục hồi và báo cáo	59e062a
Kiểm chứng	Clone mới, seed lặp và kiểm tra dependency	fab23f7

Trong phát triển, yêu cầu được phân rã thành thay đổi ở giao diện, tuyến HTTP, domain và schema. Một chức năng chỉ được coi là hoàn tất khi đường đi thành công và tình huống bị từ chối đều có kết quả rõ ràng. Ví dụ, bổ sung thao tác hoàn tất chuyến cần đồng thời xét trạng thái phiếu, kilomet cuối, cập nhật xe và kiểm thử dữ liệu không hợp lệ.

Các thư mục app, infra, observability, scripts và docs phân tách trách nhiệm. Giao diện EJS không quyết định trực tiếp việc một xe được nhận lịch; quy tắc đó nằm ở domain. Script vận hành không thay thế kiểm tra nghiệp vụ. Cách tổ chức này giúp thay giao diện hoặc bổ sung một trường dữ liệu mà vẫn xác định được nơi cần sửa và nơi phải kiểm thử lại.

package-lock.json khóa dependency và Dockerfile dùng npm ci. Các image có tag phiên bản cụ thể để dễ tái hiện, nhưng tag vẫn cần được kiểm soát khi lập bản phát hành. .env chỉ cấu hình môi trường; secret, backup và dữ liệu runtime được Git bỏ qua. Đợt kiểm tra đã ghi nhận npm audit production có 0 lỗ hổng; đây là kết quả tại thời điểm kiểm tra, không thay thế việc rà soát định kỳ.

Quy trình phát hành tiếp theo nên lưu commit, thay đổi schema, kết quả kiểm thử và hướng khôi phục trong cùng một hồ sơ thay đổi. CI/CD là hướng phát triển; bản hiện tại cung cấp script chạy và kiểm thử, chưa có pipeline GitHub Actions tự động triển khai.

CHƯƠNG 2: TRIỂN KHAI ỨNG DỤNG WEB, CƠ SỞ DỮ LIỆU VÀ REVERSE PROXY

2.1. Phân tích chức năng và tổ chức ứng dụng

Chức năng được xây dựng quanh bốn đối tượng nghiệp vụ: xe, tài xế, chuyến và phiếu bảo dưỡng. Quản trị viên xem danh sách, mở hồ sơ chi tiết, nhập biểu mẫu và nhận thông báo sau thao tác. Dữ liệu đã phát sinh lịch sử được giữ để đối chiếu; trạng thái ngừng hoạt động thay cho việc xóa hồ sơ nghiệp vụ.

Module	Trách nhiệm phát triển	Điều kiện quan trọng
Xe	Chuẩn hóa mã, biển số, loại xe và kilomet	Không giảm kilomet; không ngừng sử dụng khi còn lịch mở
Tài xế	Quản lý thông tin và trạng thái làm việc	Mã và giấy phép duy nhất; còn lịch thì chưa ngừng hoạt động
Chuyến đi	Lập lịch, đổi lịch và chuyển trạng thái	Đủ xe/tài xế hoạt động và không giao lịch mở
Bảo dưỡng	Theo dõi lịch, đơn vị làm và kết quả	Kiểm tra chuyến cùng xe; chi phí nguyên không âm
Tài khoản	Xác thực, CSRF, đổi mật khẩu và quản lý phiên	Băm mật khẩu; thay phiên khi đăng nhập; hủy phiên cũ

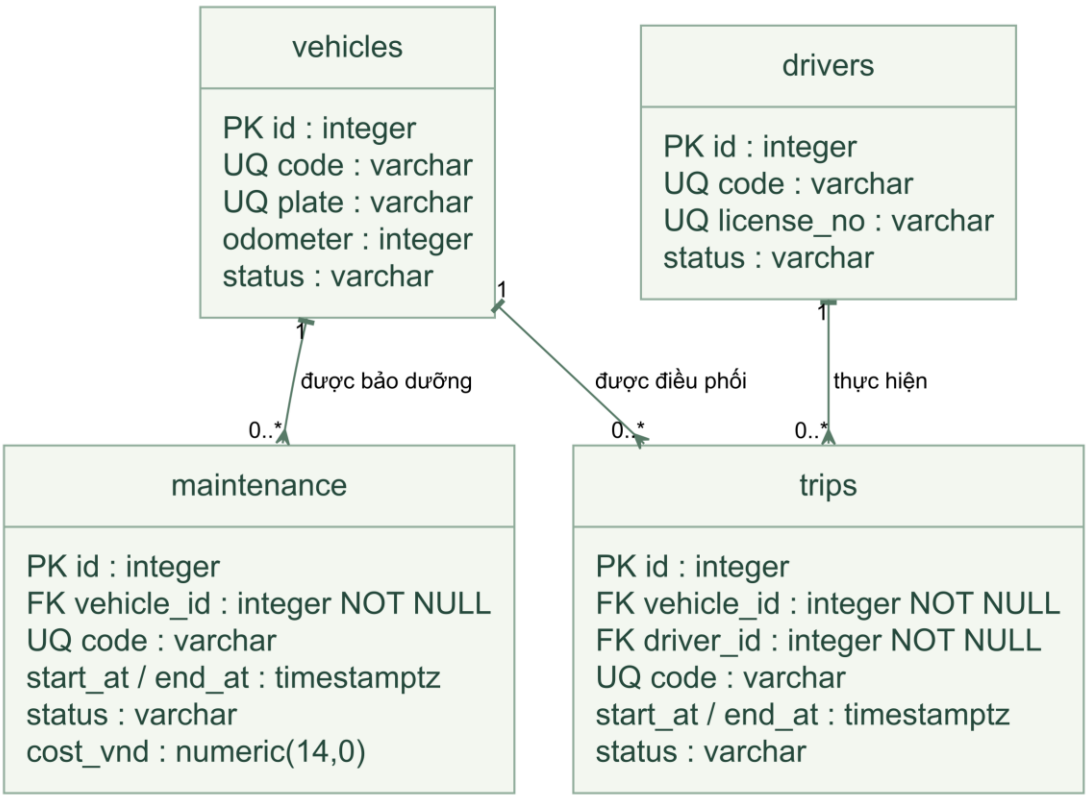
server.js tiếp nhận HTTP, kiểm tra phiên và gọi hàm nghiệp vụ. domain.js chuẩn hóa đầu vào, mở transaction, lấy khóa và thực thi điều kiện. db.js quản lý pool kết nối và đảm bảo commit hoặc rollback. Các view chỉ trình bày dữ liệu đã được truy vấn và thông báo do máy chủ quyết định, hạn chế lặp quy tắc ở nhiều màn hình.

Tách trách nhiệm đặc biệt cần thiết khi biểu mẫu bị gửi lại hoặc hai người thao tác gần như cùng lúc. Thuộc tính required trên trình duyệt giúp nhập liệu thuận tiện nhưng không đủ bảo vệ database. Máy chủ vẫn phải kiểm tra trường bắt buộc, giới hạn độ dài, định dạng, phiên bản bản ghi và trạng thái hiện tại trước khi ghi.

Tình trạng vận hành của xe được suy ra từ hồ sơ và công việc đang thực hiện. Xe hoạt động nhưng chưa có công việc đang chạy được xem là sẵn sàng; có chuyến đang chạy hoặc phiếu bảo dưỡng đang làm thì hiển thị tình trạng tương ứng. Cách suy ra này tránh tạo thêm một cột trạng thái có thể lệch khỏi lịch thực tế.

2.2. Thiết kế cơ sở dữ liệu

Thiết kế dùng ID tự sinh làm khóa chính và mã hiển thị duy nhất. Xe, tài xế, chuyến và bảo dưỡng được liên kết bằng khóa ngoại bắt buộc.



Hình 2. ERD rút gọn của bốn bảng nghiệp vụ

Xe có 0..n chuyến và 0..n phiếu bảo dưỡng; tài xế có 0..n chuyến. Mỗi chuyến gắn một xe và một tài xế; mỗi phiếu gắn một xe. Quan hệ được giữ khi hồ sơ thay thông tin.

Bảng bổ trợ	Mục đích
admins	Lưu username, password_hash và auth_version
sessions	Lưu sid, dữ liệu phiên JSON và thời điểm hết hạn
seed_versions	Đánh dấu bộ dữ liệu khởi tạo đã áp dụng

Phiên lưu JSON, không có FK vật lý tới admins. Ứng dụng kiểm tra auth_version để vô hiệu hóa phiên cũ sau khi đổi mật khẩu.

2.3. Ràng buộc dữ liệu và quy tắc kiểm tra

Thiết kế dữ liệu cần thể hiện được cả trạng thái bình thường lẫn điều kiện từ chối. Database bảo vệ kiểu dữ liệu, duy nhất và tham chiếu; domain bổ sung những điều kiện phụ thuộc nhiều bảng hoặc trạng thái hiện tại. Hai lớp này hỗ trợ nhau, không nên giả định mọi quy tắc đã có sẵn trong SQL.

Nhóm trường	Ràng buộc đã áp dụng	Ý nghĩa
code, plate, license_no	UNIQUE; chuẩn hóa mã và biển số	Tránh nhập trùng hồ sơ hoặc phiếu
vehicle_id, driver_id	NOT NULL và khóa ngoại	Mỗi lịch có tài nguyên tồn tại
start_at, end_at	timestamptz; end_at > start_at	Lưu thời điểm rõ múi giờ và khoảng hợp lệ
odometer	Số nguyên không âm; domain chặn giảm	Bảo toàn tiến trình kilomet xe
cost_vnd	numeric(14,0), không âm	Không phát sinh tiền lẻ hoặc chi phí âm
status, version	Tập trạng thái và số phiên bản	Giới hạn vòng đời; phát hiện biểu mẫu cũ

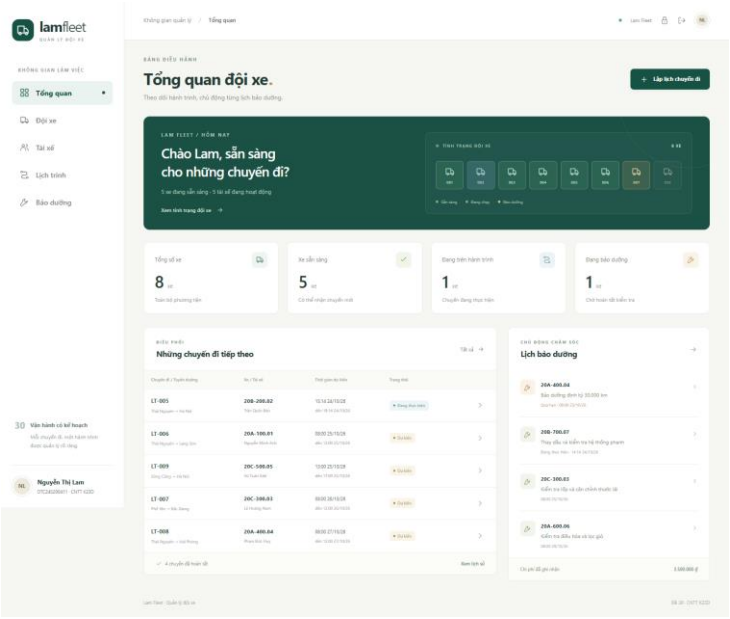
Ngày giờ từ datetime-local được hiểu là giờ Việt Nam rồi đổi về một thời điểm có múi giờ trước khi gửi PostgreSQL. Bộ kiểm tra còn đối chiếu lại ngày đã phân tích để từ chối ngày không tồn tại như 31/02. Chi phí được kiểm tra ở máy chủ thay vì tin vào min hoặc step của ô nhập trên trình duyệt.

Các chỉ mục theo vehicle_id, driver_id, start_at và end_at phục vụ truy vấn lịch mở. Chỉ mục duy nhất từng phần bảo vệ một chuyến đang chạy trên một xe/tài xế và một phiếu bảo dưỡng đang làm trên một xe. Tuy nhiên, quy tắc giao lịch giữa trips và maintenance vẫn được thực hiện trong domain dưới transaction và khóa tài nguyên.

Người quản trị database cần hiểu giới hạn này trước khi sửa SQL trực tiếp. Một thao tác bỏ qua ứng dụng có thể không đi qua kiểm tra lịch chồng. Với thay đổi dữ liệu quan trọng, cần dùng quy trình có kiểm tra trước/sau và sao lưu, thay vì coi quyền DDL/DML là bảo đảm dữ liệu sẽ tự đúng nghiệp vụ.

2.4. Phát triển giao diện và giao tiếp HTTP

Giao diện được phát triển bằng EJS và CSS thích ứng, sử dụng cùng hệ điều hướng cho tổng quan, đội xe, tài xế, lịch trình và bảo dưỡng. Máy chủ render HTML từ dữ liệu PostgreSQL; sau khi ghi thành công, trình duyệt được chuyển hướng để tải lại trạng thái đã lưu.



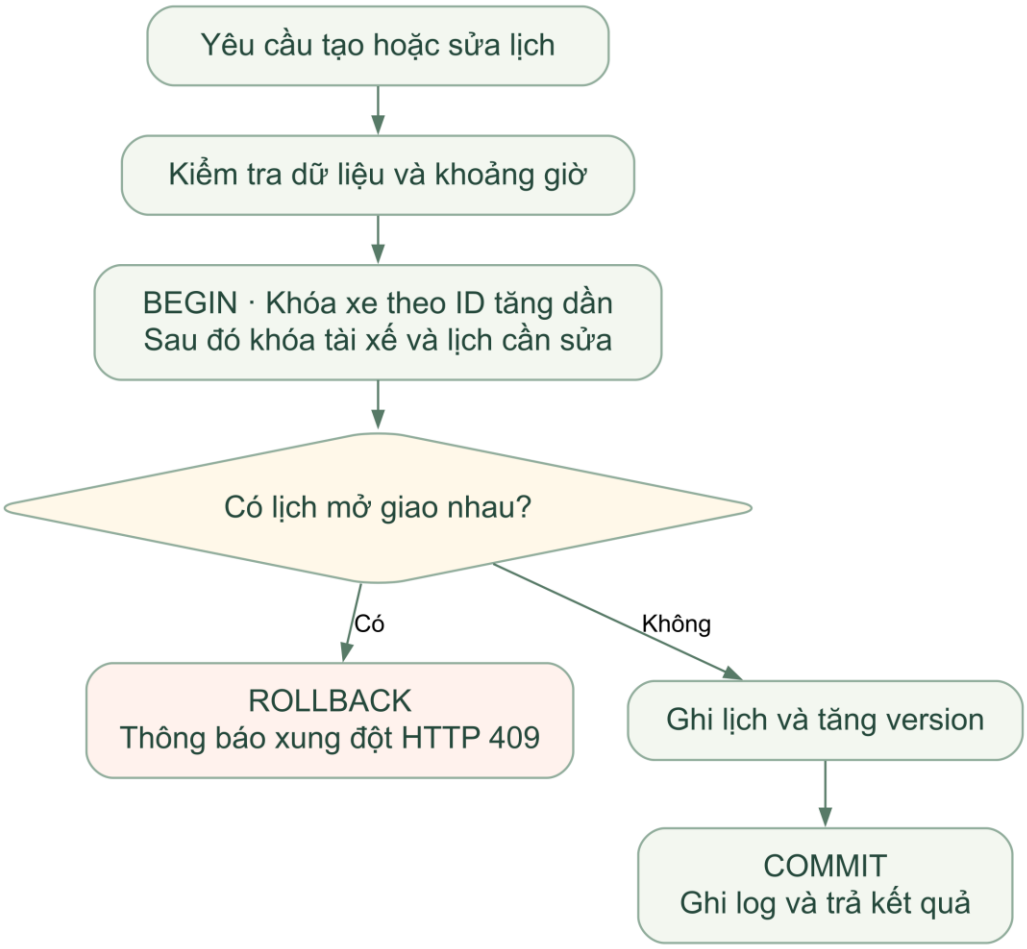
Hình 3. Giao diện tổng quan phục vụ quản trị đội xe

Nhóm đường dẫn	Hành vi giao tiếp
GET /vehicles, /drivers	Danh sách, lọc, hồ sơ chi tiết và biểu mẫu
GET /trips, /maintenance	Xem lịch, trạng thái và lịch sử nghiệp vụ
POST tạo hoặc sửa hồ sơ/lịch	Nhận form URL-encoded, kiểm tra CSRF và version
POST .../:id/:action	Bắt đầu, hoàn tất hoặc hủy theo trạng thái hiện tại
GET /metrics, /readyz	Dành cho nội bộ; Nginx website chặn truy cập

Mã 303 biểu thị ghi thành công và chuyển trang; 400 cho dữ liệu sai, 403 cho CSRF, 404 khi không tìm thấy và 409 cho xung đột hoặc bản ghi đã thay đổi. Tìm kiếm dùng tham số SQL và view xuất dữ liệu có escape. Chưa cung cấp REST API JSON công khai; đường dẫn ở đây phục vụ giao diện biểu mẫu của ứng dụng.

2.5. Transaction và xử lý đồng thời

Khoảng lịch [start, end) cho phép hai lịch tiếp giáp. Lịch 08:00–10:00 và 10:00–12:00 được chấp nhận; 09:00–11:00 trên cùng tài nguyên bị từ chối.



Hình 4. Transaction và kiểm soát tranh chấp khi ghi lịch

Transaction khóa xe theo ID tăng dần, rồi tài xế và bản ghi lịch cần sửa. Sau khóa, ứng dụng đọc lại dữ liệu và kiểm tra cả chuyến/bảo dưỡng; xung đột dẫn tới rollback và HTTP 409.

version chống biểu mẫu cũ ghi đè; khóa tài nguyên chống hai yêu cầu chiếm cùng lịch trống. Kiểm thử cạnh tranh cùng xe, cùng tài xế và chuyến với bảo dưỡng cho đúng một yêu cầu thắng.

2.6. Xây dựng vòng đời chuyến đi và bảo dưỡng

Vòng đời được biểu diễn bằng bốn trạng thái planned, in_progress, completed và cancelled. Các trạng thái không chỉ là nhãn hiển thị: chúng quyết định quyền chỉnh sửa, điều kiện dùng tài nguyên và dữ liệu bắt buộc khi hoàn tất. Tập thao tác được kiểm tra lại tại máy chủ kể cả khi nút tương ứng không xuất hiện trên giao diện.

Trạng thái hiện tại	Thao tác hợp lệ	Kết quả hoặc điều kiện
Dự kiến	Sửa, bắt đầu, hủy	Sửa cần đúng version và lịch không giao nhau
Đang thực hiện	Hoàn tất	Không cho sửa lịch hoặc hủy công việc đã bắt đầu
Hoàn tất	Xem lịch sử	Giữ kết quả, thời điểm và kilomet đã ghi
Đã hủy	Xem lịch sử	Giải phóng khoảng đặt; giữ dấu vết phiếu

Khi bắt đầu chuyến, ứng dụng kiểm tra xe và tài xế vẫn hoạt động, đồng thời kiểm tra công việc đang thực hiện trên cùng tài nguyên. start_odometer được lấy từ xe để làm mốc. Khi hoàn tất, kilomet cuối không được thấp hơn số hiện tại của xe hoặc mốc đầu chuyến; cập nhật chuyến và xe được ghi cùng transaction.

Bảo dưỡng sử dụng cùng nguyên tắc điều phối xe nhưng không cần tài xế. Hoàn tất cần kilomet, chi phí VND nguyên không âm và nội dung kết quả. Thống kê chi phí chỉ cộng phiếu hoàn tất, tránh coi dự kiến là khoản tiền đã phát sinh. Phiếu đã kết thúc không mở lại bằng chức năng chỉnh sửa thông thường.

Việc bắt đầu và hoàn tất do quản trị viên chủ động xác nhận; ứng dụng chưa tự chuyển trạng thái theo đồng hồ hoặc ép thời điểm thực tế trùng khoảng dự kiến. Nếu công việc kéo dài, tài nguyên vẫn bị chặn bắt đầu công việc khác cho đến khi hoàn tất. Người điều phối cần chủ động xử lý lại các lịch tiếp theo, vì bản hiện tại chưa tự dời lịch.

Quy tắc ngừng hoạt động bảo vệ lịch mở: xe còn chuyến hoặc phiếu bảo dưỡng chưa kết thúc thì chưa được ngừng sử dụng, tài xế còn chuyến mở thì chưa được ngừng hoạt động. Đây là cách duy trì tính liên tục của hồ sơ và ngăn lịch đã lập trỏ tới một tài nguyên không còn sẵn sàng.

2.7. Phát triển xác thực và quản lý phiên

Xác thực được xây dựng ở phía máy chủ. Mật khẩu không lưu dạng rõ mà được băm bằng scrypt với salt ngẫu nhiên; khi đăng nhập, kết quả băm được so sánh bằng cơ chế có thời gian so sánh ổn định. Secret phiên được tách khỏi mã nguồn và đọc từ file được mount vào container.

Sau khi đăng nhập thành công, session được tái tạo để không giữ định danh phiên trước xác thực. Cookie lam.fleet.sid có HttpOnly, SameSite=Lax và tuổi thọ 8 giờ. Dữ liệu phiên lưu tại PostgreSQL giúp ứng dụng không phụ thuộc vào bộ nhớ của một lần khởi động tiến trình. Cookie riêng cũng tránh nhầm phiên với các bài khác trên localhost.

Thành phần	Cách phát triển	Tác dụng và giới hạn
CSRF token	Sinh trong phiên, so sánh trên POST	Chặn yêu cầu thay đổi thiếu token hợp lệ
Giới hạn thử đăng nhập	15 yêu cầu trong 15 phút theo IP	Giảm thử lặp; cần xem xét proxy khi công khai
auth_version	Kiểm tra trên yêu cầu đã xác thực	Phiên cũ hết hiệu lực sau đổi mật khẩu
Đổi mật khẩu	Xác nhận mật khẩu hiện tại và độ mạnh	Không chỉ sửa file secret khởi tạo
Cookie Secure	Cấu hình theo môi trường	Demo HTTP để false; HTTPS cần bật true

Khi đổi mật khẩu, ứng dụng tăng auth_version và hủy phiên hiện tại. Các phiên khác bị từ chối khi phiên bản không còn khớp. Kiểm thử sử dụng tài khoản QA riêng để xác nhận hành vi này, không thay mật khẩu của admin dùng cho buổi demo. File mật khẩu khởi tạo không tự đổi theo thao tác đổi mật khẩu trên website.

Về quản trị, tài khoản website, pgAdmin, Grafana và role PostgreSQL có mục đích khác nhau. Không dùng chung một mật khẩu cho mọi lớp hoặc chia sẻ role migration như tài khoản ứng dụng. Khi mở rộng nhiều người dùng, cần bổ sung mô hình vai trò, nhật ký ai thay đổi và quy trình thu hồi quyền; những phần đó chưa được triển khai trong bản một quản trị viên.

2.8. Đóng gói ứng dụng và cấu hình reverse proxy

Dockerfile đóng gói app bằng Node.js 24, cài dependency qua npm ci và chạy tiến trình dưới user node. Compose khai báo PostgreSQL, migration, ứng dụng, pgAdmin và Nginx; hai file mở rộng bổ sung giám sát và log. Dữ liệu bền nằm trên named volume, không nằm trong lớp ghi tạm của image ứng dụng.

Thứ tự khởi động bảo đảm database có thể nhận kết nối trước khi migration chạy; app chỉ bắt đầu sau khi schema và seed sẵn sàng. Health check kiểm tra app/database và Nginx trước khi script báo hoàn tất. Nhờ tách migrate khỏi tiến trình web, nhiệm vụ thay đổi schema có quyền riêng và không phải cấp quyền DDL cho app.

Cấu hình Nginx	Hành vi thực tế
Cổng 8080 trên localhost	Cổng truy cập duy nhất của website demo
Reverse proxy app:3000	Chuyển tiếp HTTP qua mạng frontend
CSP và X-Frame-Options DENY	Giới hạn nguồn nội dung và việc nhúng trang
nosniff, Referrer-Policy	Giảm rủi ro suy đoán MIME và lộ tham chiếu
Giới hạn body và timeout	Hạn chế yêu cầu lớn hoặc treo kéo dài
/metrics, /readyz	Không công khai qua cổng website

Bản demo dùng HTTP localhost cùng security headers; chưa triển khai HTTPS tự ký hoặc cấu hình UFW riêng cho bài. Khi thay môi trường, phải rà soát tên miền, chứng chỉ, cookie Secure và quyền truy cập công cụ quản trị. Không nên sao chép ví dụ cấu hình rồi mô tả thành biện pháp đã áp dụng nếu chưa có cấu hình và kiểm chứng tương ứng.

```
sh scripts/start.sh
sh scripts/test.sh
sh scripts/stop.sh
```

PowerShell wrapper gọi đúng Docker Engine trong Ubuntu WSL2 và giữ phiên WSL tối đa 4 giờ. Khi dùng lệnh Linux trực tiếp, cần giữ phiên Ubuntu hoạt động trong buổi demo. Dừng thông thường giữ volume; khởi động lại không phải thao tác khôi phục backup và không được dùng làm lý do xóa dữ liệu.

CHƯƠNG 3: GIÁM SÁT HỆ THỐNG VỚI PROMETHEUS, GRAFANA VÀ LOKI

3.1. Quản trị giám sát bằng Prometheus và Grafana

Giám sát phục vụ việc phát hiện và khoanh vùng vấn đề, không thay thế dữ liệu nghiệp vụ. Prometheus thu thập mỗi 15 giây từ năm job prometheus, lam-fleet, nginx, postgres và cadvisor. App cung cấp metrics HTTP và trạng thái được đọc từ PostgreSQL; exporter và cAdvisor bổ sung góc nhìn web server, database và container.

Nhóm số liệu	Câu hỏi quản trị	Cách đối chiếu
Target và exporter	Thành phần có đang thu thập được không?	up, nginx_up, pg_up
CPU và RAM	Tài nguyên đang tập trung ở app, Nginx hay DB?	Lọc project de30lam và từng service
HTTP và p95	Lỗi hoặc độ trễ tăng ở giai đoạn nào?	Mã phản hồi, histogram độ trễ
Kết nối PostgreSQL	Ứng dụng có tạo áp lực lên database không?	Số backend, tốc độ transaction
Trạng thái nghiệp vụ	Chuyến và bảo dưỡng đang phân bổ ra sao?	Gauge đọc dữ liệu bền trong DB

Dashboard Grafana có 14 panel metrics và 3 panel log, được provision từ mã nguồn. Mỗi truy vấn container lọc đúng nhãn de30lam để không lấy nhầm số liệu bài khác cùng Docker host. Kết quả đã ghi nhận 5 target UP và CPU/RAM thực của app, nginx, postgres; chỉ nhìn thấy Prometheus hoạt động chưa đủ để kết luận giám sát hoàn chỉnh.

Khi diễn giải biểu đồ, cần phân biệt counter với gauge. Counter HTTP/thao tác khởi động lại cùng app, còn gauge xe và trạng thái đọc database nên phản ánh dữ liệu vẫn tồn tại sau restart. Biểu đồ rate cần tối thiểu hai lần scrape. Dữ liệu được giữ 7 ngày và có giới hạn dung lượng theo cấu hình demo.

Quy trình xử lý cảnh báo nên bắt đầu từ target, sau đó kết nối database, tài nguyên container và lỗi ứng dụng trong cùng khoảng thời gian. Các alert đã khai báo trong Prometheus chưa được nối với email hay kênh thông báo ngoài. Việc bổ sung ngưỡng dựa trên tải thực và tuyến thông báo là công việc quản trị tiếp theo, không phải kết quả đã có trong bản này.

3.2. Quản lý log và truy vết sự cố

Log tập trung ghi nhận diễn biến cụ thể để giải thích sự thay đổi trên metrics. Ứng dụng và Nginx ghi JSON; PostgreSQL ghi JSON log trên volume riêng. Promtail đọc file ở chế độ chỉ đọc, lưu vị trí đọc và đẩy sang Loki. Cách thu thập này chỉ áp dụng các stream được cấu hình cho Lam Fleet, không tự đọc mọi container trên máy.

```
{job="nginx", project="de30lam"}
  | json | status >= 400
```

Truy vấn trên tìm các yêu cầu HTTP bị từ chối hoặc lỗi. Mã 400/403/409 trong thử nghiệm có thể là kết quả đúng của kiểm tra dữ liệu, CSRF hoặc trùng lịch; không nên coi mọi phản hồi 4xx là lỗi chương trình. Cần đối chiếu tình huống người dùng và request_id trước khi kết luận.

```
{job="app", project="de30lam"} | json
  | event=~"(trip|maintenance)_(scheduled|started|completed|cancelled)"
```

Nhóm sự kiện nghiệp vụ cho biết lịch được tạo, bắt đầu, hoàn tất hoặc hủy. Request ID giúp liên kết lỗi HTTP với log app trong cùng thao tác. Log có thể hỗ trợ điều tra, nhưng lịch sử nghiệp vụ bền vẫn nằm trong database; thời gian giữ log ngắn hơn thời gian lưu hồ sơ.

```
{job="postgres", project="de30lam"} | json
  | error_severity=~"ERROR|FATAL|PANIC"
```

Lỗi PostgreSQL cần được phân loại theo nguyên nhân, chẳng hạn mã trùng trong kiểm thử hay kết nối bị kết thúc khi dịch vụ dừng. Bộ kiểm chứng đã xác nhận cả ba truy vấn có sự kiện. Truy vấn và kết quả một lần chạy được giữ trong hồ sơ minh chứng để có thể đối chiếu lại, thay cho việc chèn nhiều ảnh giao diện vận hành.

Mật khẩu, cookie và nội dung form không được đưa vào log ứng dụng. File được xoay vòng và Loki giữ dữ liệu 7 ngày. Promtail được giữ theo yêu cầu học phần; công cụ đã hết hỗ trợ từ 02/03/2026 theo Grafana [4], nên kế hoạch nâng cấp cần chuyển sang Alloy hoặc client còn được hỗ trợ. Tính tương thích cấu hình và nhãn log phải được kiểm tra khi chuyển đổi.

CHƯƠNG 4: CÁC BIỆN PHÁP HARDENING VÀ ĐÁNH GIÁ BẢO MẬT

4.1. Phân quyền và gia cố bảo mật

Hardening được xem xét theo đặc quyền cần cho từng trách nhiệm, không chỉ theo số lượng tùy chọn Docker được bật. Chạy non-root làm giảm quyền của tiến trình khi bị khai thác, nhưng không bảo đảm ngăn tuyệt đối mọi hình thức leo thang đặc quyền. Cấu hình phải đi kèm kiểm tra phiên bản, mount, network và công cụ quản trị.

Lớp kiểm soát	Biện pháp đã cấu hình	Phạm vi xác nhận
App và Nginx	Non-root, read-only rootfs, bỏ capability, no-new-privileges	Đọc container inspect
Database	Tách owner, migrate, app và exporter	Kiểm tra quyền DDL/DML
Network	Database nội bộ; không publish app, DB và exporter	Kiểm tra network/port
Giao diện	Cookie, CSRF, scrypt, SQL tham số hóa và headers	Source và các ca HTTP liên quan
Secret	Sinh riêng; thư mục 0700; không commit	Filesystem và Git
Log	Mount chỉ đọc, không log credential	Cấu hình và stream đã thu

fleet_app có quyền đọc tài khoản phục vụ xác thực, cập nhật mật khẩu/auth_version, thao tác phiên và ghi dữ liệu nghiệp vụ. Role này không có CREATE trên schema public và không được DELETE lịch sử trips/maintenance. Quyền DELETE trên sessions vẫn cần thiết cho việc hủy phiên; vì vậy mô tả mọi quyền xóa đều bị cấm là không đúng cấu hình.

Ngoại lệ có chủ đích gồm cAdvisor privileged để đọc host/Docker, storage-init dùng root tạm thời để đặt quyền volume, PostgreSQL bootstrap root rồi chuyển postgres và logrotate có capability hẹp cho file log. Các ngoại lệ này phải được quản trị như thành phần có độ tin cậy cao, thay vì che giấu trong nhận định tất cả container đều không có đặc quyền.

Bản hiện tại chỉ bind các cổng giao diện vào 127.0.0.1 và phục vụ demo nội bộ. Chưa có kiểm thử xâm nhập toàn diện, cấu hình firewall riêng hay đánh giá đáp ứng tải lớn.

4.2. Quản trị dữ liệu và sao lưu phục hồi

Quản trị dữ liệu bắt đầu từ phân biệt quyền ứng dụng và quyền quản lý schema. pgAdmin dùng role migration trong môi trường demo; tài khoản này có quyền lớn hơn fleet_app và không nên được dùng để chạy ứng dụng. Secret đăng nhập pgAdmin và secret kết nối PostgreSQL được sinh riêng, không nằm trong repository.

Migration tạo schema và seed trong transaction, sử dụng advisory lock để tránh khởi tạo đồng thời. seed_versions ghi nhận bộ dữ liệu đã áp dụng. Kiểm tra trên clone mới đã xác nhận chạy lại migration không nhân đôi dữ liệu và không đặt lại mật khẩu admin. Việc cập nhật schema về sau cần có script migration có phiên bản, không chỉ sửa trực tiếp database đang dùng.

```
sh scripts/backup.sh
sh scripts/restore.sh --verify-current .local/backups/<tep>.dump
```

Backup dùng pg_dump dạng custom, lưu schema, dữ liệu, ownership và ACL, kèm SHA-256 để kiểm tra file. Archive chứa tài khoản và phiên nên được giữ trong thư mục hạn chế quyền, không đưa vào Git như minh chứng. Thời gian giữ bản sao và bản lưu ngoài máy cần được quy định khi sử dụng thực tế; bản demo chưa tự động hóa các chính sách đó.

Chế độ	Hành vi quản trị
--verify	Khôi phục vào database tạm để kiểm tra archive
--verify-current	Kiểm tra thêm fingerprint bốn bảng khớp dữ liệu hiện tại
--apply	Xác nhận, dừng app/Nginx rồi thay dữ liệu hiện có

Kết quả kiểm tra đã phục hồi khớp 8 xe, 6 tài xế, 10 chuyến và 5 phiếu trong database tạm. Fingerprint dùng MD5 để đối chiếu nội dung nghiệp vụ, không dùng như cơ chế bảo mật; kiểm tra toàn vẹn archive dùng SHA-256. Chế độ --apply đã có trong script nhưng không được thực hiện ghi đè database demo ở đợt nghiệm thu.

Nếu apply thất bại, app và Nginx giữ trạng thái dừng để xử lý. Mật khẩu sau phục hồi theo dữ liệu trong backup; file mật khẩu khởi tạo không được tự viết lại. Trước khi cho người dùng truy cập trở lại cần kiểm tra kết nối, quyền, dữ liệu và một thao tác nghiệp vụ, không chỉ kiểm tra container đã chạy.

4.3. Quản lý thay đổi và xử lý sự cố

Một thay đổi hạ tầng hoặc schema cần có mục tiêu, phiên bản nguồn, ảnh hưởng dự kiến, cách kiểm chứng và phương án quay lại. Tách cấu hình môi trường khỏi mã giúp chuyển máy mà không sửa nghiệp vụ, nhưng người quản trị vẫn phải kiểm tra đúng Compose project, secret và volume trước mỗi thao tác có thể thay dữ liệu.

Giai đoạn	Việc quản trị cần thực hiện	Điều kiện hoàn tất
Trước thay đổi	Chốt commit, kiểm tra cấu hình và lưu backup phù hợp	Có phiên bản và bản khôi phục xác định
Thực hiện	Build, chạy migration và khởi động đúng project	Không tác động volume bài khác
Xác nhận	Kiểm tra HTTP, DB, metrics, log và nghiệp vụ	Đạt kiểm tra theo phạm vi thay đổi
Kết thúc	Lưu kết quả, ghi hạn chế và cập nhật hướng dẫn	Người nhận có thể tái hiện

Khi website không truy cập được, cần kiểm tra phiên WSL, Docker Engine, cổng 8080 và trạng thái dịch vụ. Nếu Nginx hoạt động nhưng app lỗi, chuyển sang health check, log app/migrate và khả năng kết nối PostgreSQL. Cổng bị chiếm phải xác định dịch vụ đang dùng; không tự dừng bài khác hoặc đổi cổng web đã chốt để che lỗi.

Khi mất biểu đồ, kiểm tra target, exporter, nhãn project và khoảng thời gian trước khi thay cấu hình. Khi thiếu log, kiểm tra quyền file, Promtail, Loki và sự kiện mới. Một sự cố có thể xuất phát từ cấu hình hoặc thiếu thời gian thu thập chứ chưa chắc là mất dữ liệu. Việc chẩn đoán theo từng lớp giúp tránh xóa volume không cần thiết.

Dừng thường xuyên giữ named volume. Không dùng docker system prune hoặc shutdown toàn WSL như thao tác sửa riêng Lam Fleet. Mật khẩu role PostgreSQL đã lưu trong database không tự đổi khi người quản trị sửa file secret; thay credential cần được thực hiện đồng bộ bằng quy trình kiểm soát và kiểm tra lại các client liên quan.

Hồ sơ bàn giao cần phân biệt bản nguồn chính trong WSL, bản ZIP chụp từ Git và tài liệu Word/PDF. Cần phân loại thay đổi tài liệu, giao diện, nghiệp vụ và schema để chọn phạm vi kiểm chứng. Cách phân loại này giới hạn tác động không cần thiết và giữ tính nhất quán giữa tài liệu, dữ liệu và phiên bản nguồn.

4.4. Kiểm thử và đánh giá kết quả

Kết quả nghiệm thu đã có 29/29 ca đạt, gồm kiểm tra dữ liệu, HTTP qua Nginx và PostgreSQL thật. Tài khoản QA và tiền tố dữ liệu riêng giúp thực hiện thử nghiệm mà không đổi mật khẩu admin chính; dữ liệu thử được dọn sau mỗi lần chạy. Các tệp tests.tap, fresh-tests.tap và kết quả JSON được lưu trong repository để đối chiếu điều kiện và kết quả kiểm chứng.

Nhóm kiểm thử	Số ca	Kết quả
Xác thực, CSRF, cookie, headers và đổi mật khẩu	5	Đạt
Màn hình và tìm kiếm tham số hóa	2	Đạt
Dữ liệu, trạng thái, quyền DB và seed	6	Đạt
Vòng đời chuyến đi	5	Đạt
Bảo dưỡng và tài nguyên đang bận	3	Đạt
Tranh chấp đồng thời	4	Đạt
Validation mã, số và ngày giờ	4	Đạt
Tổng cộng	29	29 đạt, 0 lỗi

Bốn ca đồng thời xét việc hai yêu cầu cùng đặt xe, cùng đặt tài xế, chuyến cạnh tranh với bảo dưỡng và hai lần bắt đầu cùng lịch. Kết quả có đúng một yêu cầu thành công trong từng tình huống đã thử. Các ca bổ sung xác nhận lịch tiếp giáp được phép, dữ liệu sai không tạo phiếu một phần và lịch sử hoàn tất không bị chỉnh sửa lại.

Clone riêng từ GitHub với volume mới cũng đạt 29/29 ca; chạy migration thêm lần nữa giữ số dòng seed và mật khẩu. Kiểm tra Chrome ghi nhận các màn hình desktop/mobile không lỗi JavaScript và không tràn ngang toàn trang. Về vận hành, 5 target giám sát UP, ba truy vấn LogQL có dữ liệu và backup khôi phục khớp nội dung nghiệp vụ.

Những kiểm chứng này hỗ trợ kết luận ứng dụng đáp ứng phạm vi bài thực hành. Chưa có phép thử hiệu năng tải lớn, kiểm thử xâm nhập đầy đủ, phục hồi khi mất toàn bộ máy chủ hoặc nhiều người dùng thật. Các hoạt động đó cần môi trường, dữ liệu và tiêu chí riêng trước khi mở rộng triển khai.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Lam Fleet đã chuyển yêu cầu quản lý đội xe thành các module có trách nhiệm rõ ràng, mô hình dữ liệu có ràng buộc và quy trình nghiệp vụ được kiểm tra tại máy chủ. Điểm quan trọng của phần phát triển là sự nhất quán giữa lịch dự kiến, trạng thái thực hiện, kilomet và lịch sử, cùng cơ chế xử lý khi nhiều yêu cầu cạnh tranh tài nguyên.

Ở góc độ quản trị, mã nguồn, cấu hình, quyền truy cập, secret, log và bản sao lưu cần được quản lý như các phần liên quan của cùng một hệ thống. Khả năng dựng lại từ Git và kiểm tra archive có giá trị hơn việc chỉ quan sát một lần hệ thống chạy thành công. Các hạn chế còn lại được xác định theo đúng chức năng và biện pháp đã triển khai.

Ưu tiên phát triển	Hướng thực hiện tiếp theo
Nghiệp vụ	Nhắc bảo dưỡng theo kilomet và xử lý chuyến kéo dài ảnh hưởng lịch sau
Quản trị truy cập	Nhiều tài khoản, phân quyền và truy vết người thực hiện
Triển khai	HTTPS, kiểm soát công cụ quản trị và quy trình phát hành có rollback
Quan sát và dữ liệu	Chuyển Promtail sang Alloy, sao lưu ngoài máy và diễn tập phục hồi
Tự động hóa	CI/CD kiểm tra schema, test và duyệt bản phát hành trước triển khai

Các hướng trên là đề xuất cho giai đoạn sau, không được tính như chức năng đã hoàn thành. Khi triển khai tiếp cần ưu tiên tình huống sử dụng cụ thể, tiêu chí chấp nhận và kiểm chứng tương ứng, thay vì chỉ bổ sung công cụ hoặc mở rộng số lượng container.

Tài liệu tham khảo

[1] Đề tài thực hành môn Triển khai và Quản trị Hệ thống Phần mềm K23, yêu cầu chung và đề 30.

[2] Docker Compose. https://docs.docker.com/compose/

[3] PostgreSQL 17 Explicit Locking. https://www.postgresql.org/docs/17/explicit-locking.html

[4] Grafana Promtail agent. https://grafana.com/docs/loki/latest/send-data/promtail/

[5] Prometheus Overview. https://prometheus.io/docs/introduction/overview/